



First Week

NLP internship

Table of content

First part : Important knowledge

- What is NLP (1 hour study)
- What are LLMs (2 hour study)
- Text preprocessing (5 hours study)
- RNN (5 hours study)
- LSTM (5 hours study)
- Transformers (2 days study)
- Fine tuning (1 day study)

Second part : Tasks

- Task 0
- Task 1
- Task 2
- Bonus Task

What is NLP ?

Natural Language Processing (NLP) is a branch of artificial intelligence and computational linguistics focused on enabling computers to understand, interpret, generate, and interact with human languages in a way that is meaningful and useful. It combines knowledge from linguistics, computer science, mathematics, and statistics to process text or speech data and extract patterns or meaning. NLP tasks include tokenization, part-of-speech tagging, named entity recognition, sentiment analysis, machine translation, summarization, and question answering. It relies on techniques ranging from simple rule-based methods to advanced machine learning and deep learning models such as transformers. NLP is used in applications like virtual assistants, chatbots, search engines, grammar checkers, language translation tools, and information retrieval systems. Its development involves challenges such as ambiguity, context understanding, sarcasm detection, and handling multiple languages or dialects and generating human language.

Watch these videos to Know more about NLP :

[1] Intro to Natural Language processing

https://youtu.be/1hDaLmjGDKQ?si=TUqE_7rPffTZoDsq

[2] History of Natural Language processing

https://youtu.be/uHsFiq-4-U?si=JLzRq_Ars924YaIU

[3] Applications of Natural Language processing

<https://youtu.be/igQbpL2166U?si=eaN3cZuIrK6JbmBs>

What are LLMs ?

What is ChatGPT ?

ChatGPT is an advanced conversational AI system developed by OpenAI that uses large language models (LLMs) to understand and generate human-like text., trained on massive amounts of text data to learn grammar, facts, reasoning patterns, and context. ChatGPT can answer questions, explain concepts, generate ideas, write essays or code, summarize content, translate languages, and even simulate conversation styles. It works by predicting the most probable next word or sequence of words based on the input it receives, using deep neural networks with billions of parameters to capture complex language patterns. ChatGPT can follow instructions (as an instruction-tuned model) and maintain context across a conversation, which makes it useful for tasks like tutoring, brainstorming, or technical assistance. It does not have consciousness, emotions, or independent thought—it operates purely on patterns learned from data and does not access the internet in real time unless connected to tools that allow web browsing. Its capabilities depend on the model version, with newer versions like GPT-5 offering improved reasoning, factual accuracy, and language understanding compared to earlier versions.

How ChatGPT or any LLM works

An LLM is type of AI that has learned to model the Structure and meaning of Human Language, it is system trained on massive amounts of text to predict the next word (Token) in A sequence

Example : The sky is

LLM predicts possible words :

“ Blue “ $\rightarrow$ probability = 0.87 (The highest probability)

“ Green “ $\rightarrow$ probability = 0.05

“ Falling “ $\rightarrow$ probability = 0.03

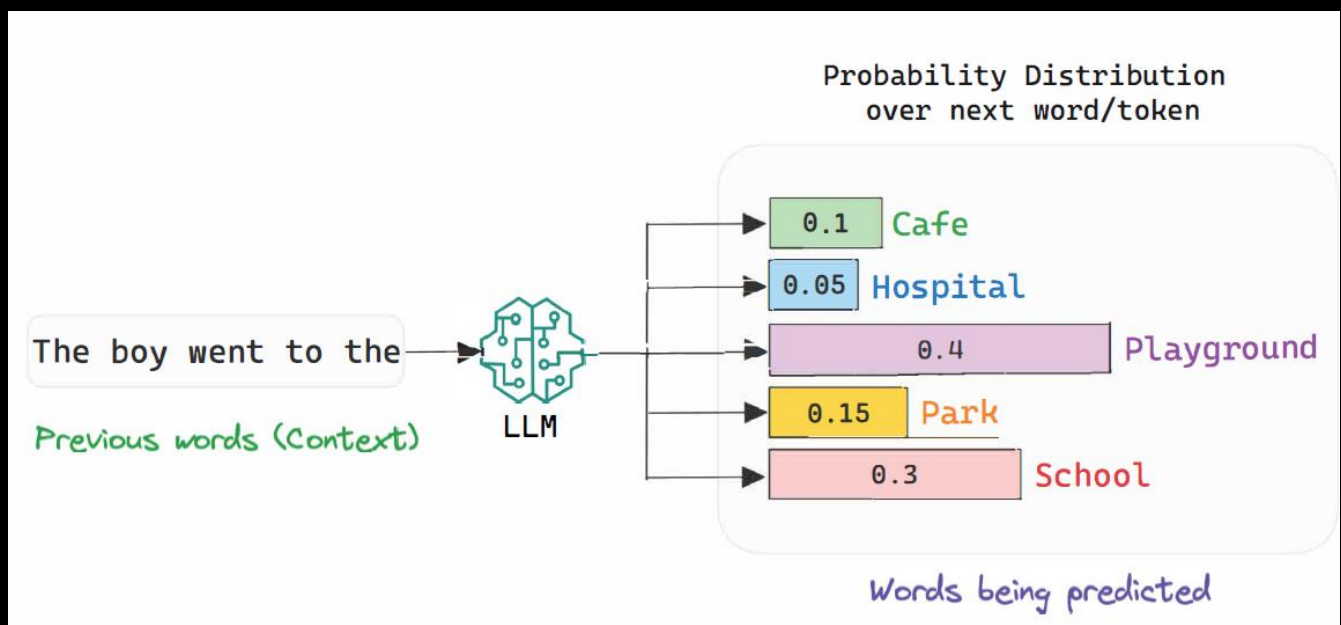
The model doesn't know what sky is like a human does, it has learned from patterns in Billions of Text Examples

$$x = [w_1, w_2, w_3, w_4, w_5, \dots, w_{t-1}]$$

Where x is sequence of tokens, w_t is the next token

$$P_{\theta}(w_t | w_1, w_2, \dots, w_{t-1})$$

Where θ are the model parameters (weights & biases)



In simple terms, any LLM does not truly understand the meaning of context or words like humans do; it is merely a statistical model that predicts the next token or word

Watch these videos to know more about ChatGPT and how it works in details :

[1] what is difference between Base LLM and Instruction Tuned LLM (watch both)

<https://youtu.be/kllhmJ2fMKA?si=8whC8Q0hkDCNJ1Ts>

<https://youtu.be/QnA4z-SxaIY?si=FOV4XAVS0cRGxSKE>

[2] what are LLMs ? (watch any)

https://youtu.be/zikBMFhNj_g?si=A28tFfXC3qqV-nu2

<https://youtu.be/zizonToFXDs?si=Fd7pMSUQUxkQFJWG>

Text Preprocessing

Watch these videos to know more about Text preprocessing and how to use these techniques in our tasks

Tokenization

<https://youtu.be/nL-DaaYHosA?si=LQ48sPVmfsAQOhgE> (independent)

<https://youtu.be/Tlw-MMJRSeE?si=lwchdZZdO3idOHIR> (part 1)

<https://youtu.be/xt3sK1QZLCM?si=0sdaAhUiNQZVJmJ2> (part 2)

https://youtu.be/xt3sK1QZLCM?si=FXS4Og2Y0b_OMm3j (part 3)

Stop words

<https://youtu.be/--nDqywVxtA?si=bKNqj8UMOVTLsRed> (independent)

<https://youtu.be/1NNNufVAkX4?si=hv-dZkJangDG-XV> (part 1)

<https://youtu.be/1AhCa5As40Y?si=3mGW5Q2NDQEzABsu> (part 2)

Stemming

<https://youtu.be/V6ou1Da9A7U?si=C87sVChLCoi4111Y>

Lemmatization

https://youtu.be/m1aUFVY17dw?si=kWsa_goArmMG5n_G

part of speech

<https://youtu.be/oSw6thswxnc?si=vR3WibmJTMrwrfeG> (independent)

https://youtu.be/-CYBZvNupUs?si=MJi7lZjU_3RIllVv (part 1)

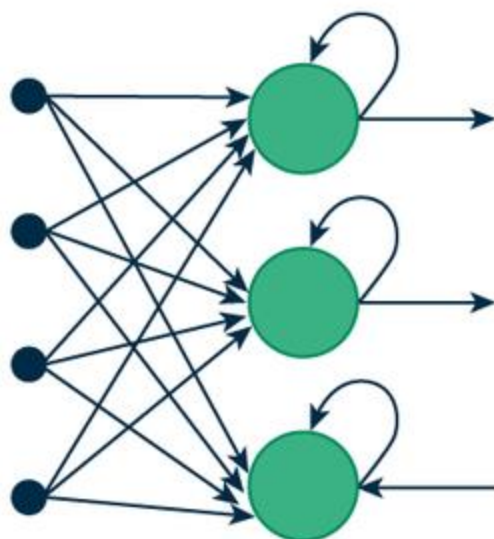
https://youtu.be/Anipp7Hlw_c?si=A5rp3Gzou8xBtlb1 (part 2)

<https://youtu.be/bbh-K8WSTq0?si=6n58TJXg56pYP6za> (part 3)

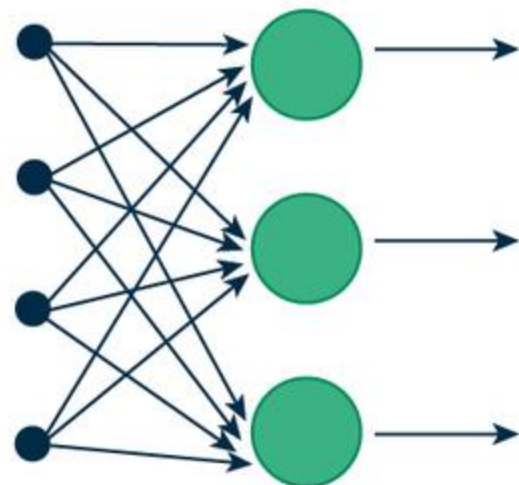
https://youtu.be/_j7Lz--7wR0?si=43VIL0XUp7VUABjd (part 4)

https://youtu.be/gX_Mbe8rx-k?si=YOOpQqh_AaVV8RDS (part 5)

RNN (Recurrent Neural Network)

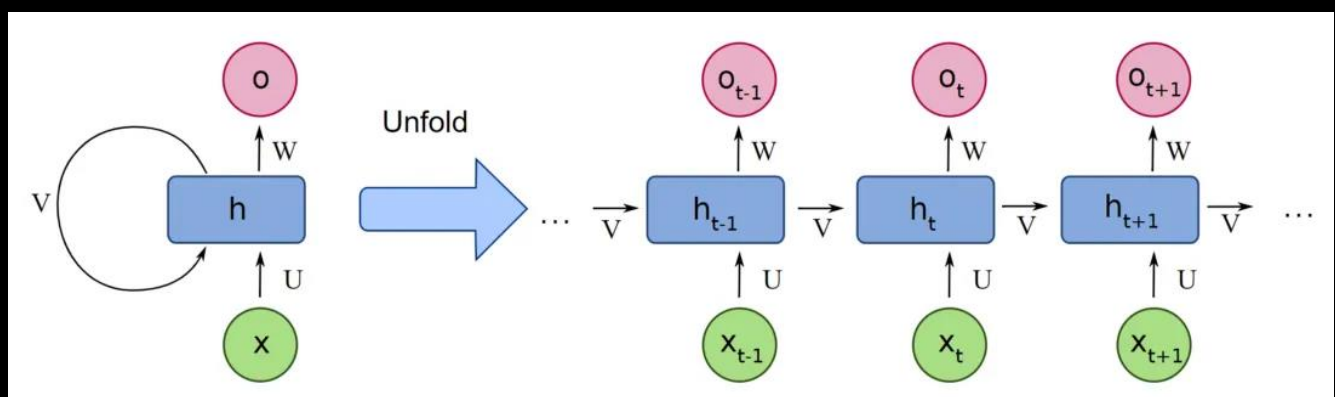


(a) Recurrent Neural Network



(b) Feed-Forward Neural Network

Recurrent Neural Networks (RNNs) are a type of artificial neural network specifically designed for processing sequential data, where the order of inputs matters. Unlike feedforward neural networks, which treat each input independently, RNNs have a feedback loop that allows information to persist across time steps. This means they maintain a hidden state—a kind of short-term memory—that captures information from previous inputs and uses it to influence the processing of current inputs. RNNs are particularly suited for tasks like natural language processing, speech recognition, time-series prediction, and sequence generation because these tasks involve data where earlier elements affect later ones. The core idea is that at each time step, an RNN takes the current input and the previous hidden state, applies weights and activation functions, and produces an output and a new hidden state. However, standard RNNs suffer from vanishing and exploding gradient problems when dealing with long sequences, making it difficult for them to learn long-term dependencies



Read this to know more about RNNs

<https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/>

Watch these videos to know more about RNNs

Playlist 1

https://youtu.be/qWcTretvSVQ?si=g04T4Jp_3e5cgwhv

playlist 2

https://youtu.be/KR5_SxZGiQ?si=b1uYUvnp6MGccBYK (1)

<https://youtu.be/Ht9vQP7jwYQ?si=K0a3m89v746iXDrN> (2)

https://youtu.be/7Gsm_loKKYM?si=faJv8QIElwxzETVX (3)

https://youtu.be/LVoZ6IASmgc?si=j82fl8SSkfhL_nCo (4)

<https://youtu.be/NLGuwx2mLPM?si=KiOz1wf-i5NluyjR> (5)

https://youtu.be/giwuoPvtjHY?si=zHDEkqwJDuf-p_zx (6)

<https://youtu.be/lnd2ISj-Rxg?si=WFprje9I6lweKBWU> (7)

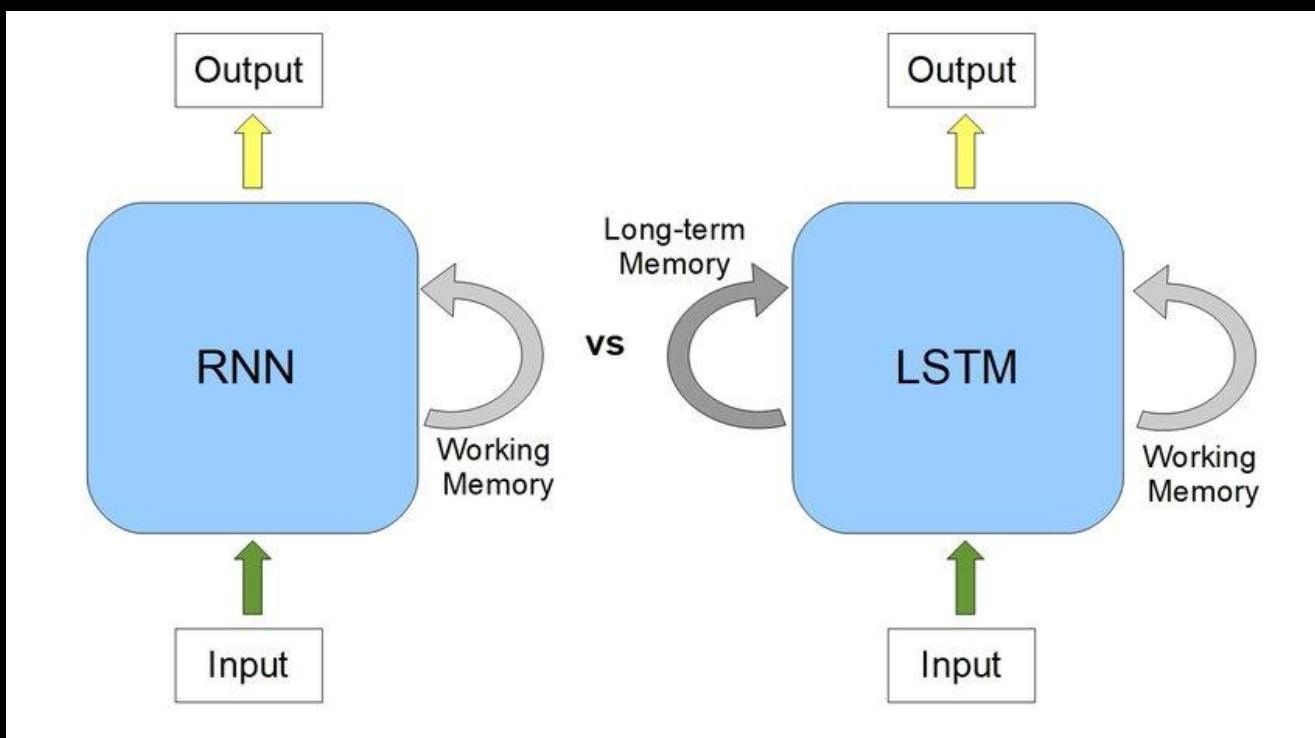
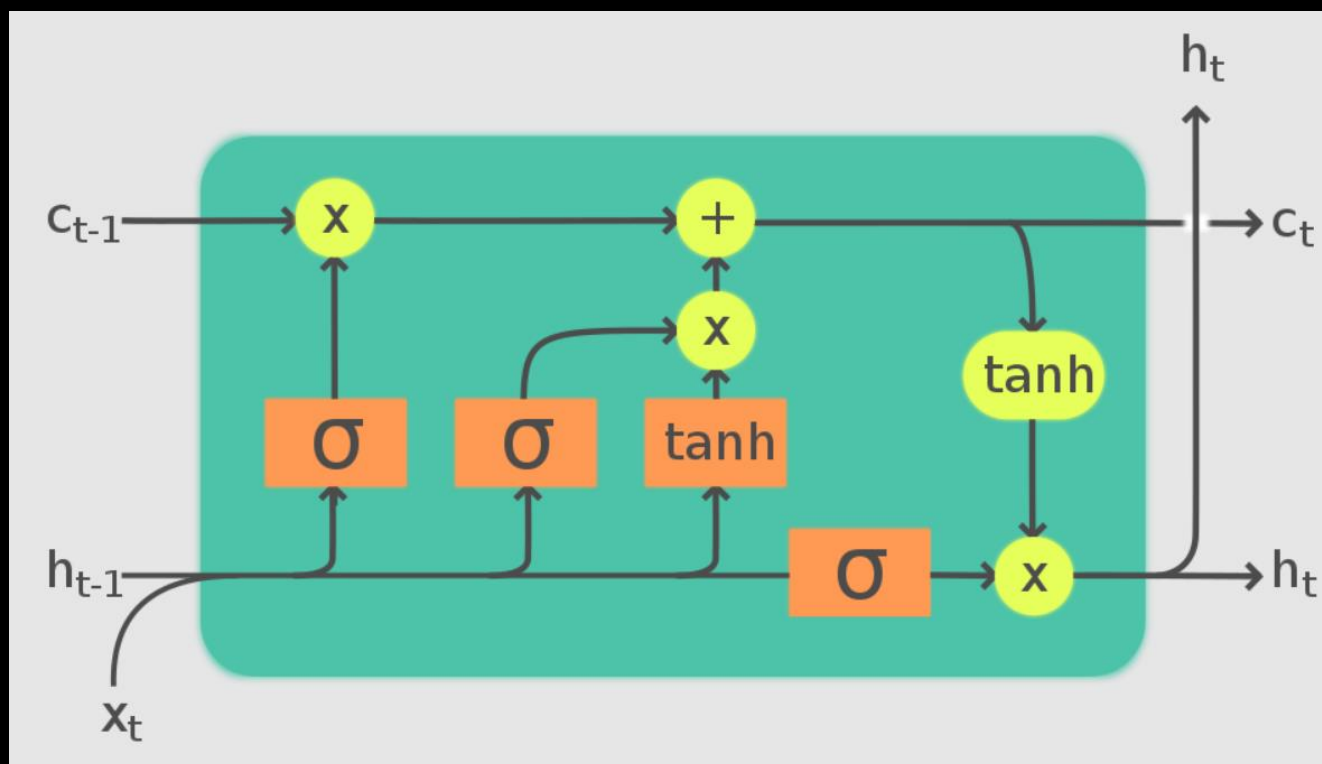
<https://youtu.be/MRVUAW7yS1M?si=nFQY6f1qc1BTuman> (8)

<https://youtu.be/9g5PlanMQTs?si=G2eDWzE0ECkdvbzz> (9)

playlist 3

https://youtu.be/rWGM2KRSPtG?si=ygjbFUdv6-1_En7x

LSTM (Long Short Term Memory)



Long Short-Term Memory (LSTM) is a specialized type of Recurrent Neural Network (RNN) designed to overcome the limitations of standard RNNs when learning long-term dependencies in sequential data. Traditional RNNs struggle with the vanishing and exploding gradient problems, which make it hard to retain information over many time steps. LSTM solves this by introducing a memory cell and three gates—input, forget, and output gates—that regulate the flow of information. The memory cell acts as long-term storage, while the gates control what information to add, remove, or output at each time step. Mathematically, the input gate decides which values from the current input and previous hidden state should be added to the cell state; the forget gate determines which parts of the cell state to erase; and the output gate decides what information from the cell state to send as the hidden state for the next step. These mechanisms allow LSTMs to keep relevant information for long periods while discarding irrelevant data. LSTMs are widely used in tasks like language modeling, speech recognition, machine translation, time-series forecasting, and video analysis. They were a major advancement in deep learning for sequence modeling before transformers became the dominant architecture.

Read this to Know more about LSTM

<https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>

Watch these videos to know more about LSTM

Playlist 1

https://youtu.be/tFVNi8QXm94?si=jRkvIB_-cBX66277 (1)

https://youtu.be/RvvQSYepaBg?si=eY_6c9tgKUROfuxZ (2)

<https://youtu.be/--iTXsFFHdo?si=l8p5w-gt7xZUJ47I> (3)

playlist 2

https://youtu.be/s2x_NMayknc?si=AqlzFoey1mDMr71d

playlist 3

<https://youtu.be/YCzL96nL7j0?si=CGDvZXURLanyS4Z4>

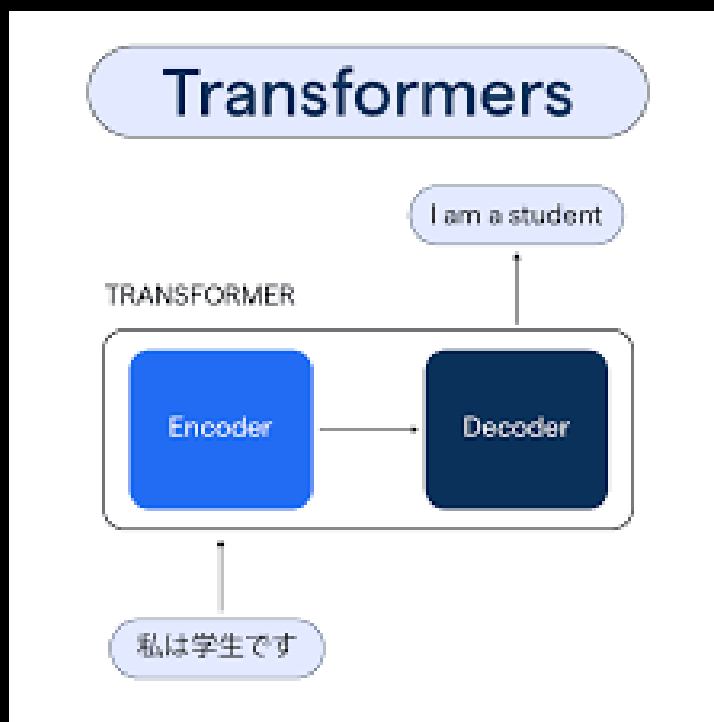
Transformers

Transformers are a deep learning architecture introduced in 2017 by Vaswani et al. in the paper “Attention Is All You Need,” and they revolutionized natural language processing and many other AI fields. Unlike RNNs or LSTMs, which process data sequentially, transformers handle entire sequences in parallel, making them more efficient and better at capturing long-range dependencies. The core innovation of transformers is the self-attention mechanism, which allows the model to weigh the importance of different tokens in a sequence relative to each other, regardless of their positions.

A transformer consists of an encoder-decoder structure (for tasks like translation) or sometimes only the encoder (like BERT) or decoder (like GPT).

Each encoder layer contains multi-head self-attention blocks and feed-forward neural networks, along with layer normalization and residual connections for stability. Positional encoding is added to input embeddings to provide sequence order information, since transformers do not inherently process tokens in order.

The decoder also uses masked self-attention to prevent future tokens from influencing current predictions, enabling autoregressive generation.



Transformers are highly scalable, support parallel computation, and have become the foundation of state-of-the-art models like BERT, GPT, T5, and Vision Transformers (ViTs). Their ability to learn contextual relationships across very large datasets has made them dominant in NLP, computer vision, speech processing, and multimodal AI applications.

The original paper link

<https://arxiv.org/pdf/1706.03762>

Read these to know more about Transformers

<https://www.geeksforgeeks.org/machine-learning/getting-started-with-transformers/>

<https://www.datacamp.com/tutorial/how-transformers-work>

[https://en.wikipedia.org/wiki/Transformer \(deep learning architecture\)](https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture))

watch these videos to know more about Transformers

playlist 1

<https://youtu.be/vsElpi4OZWc?si=OoOilN8aDfIT19C2>

<https://youtu.be/U-1IA7GsdW0?si=EyTBqfe1UCFD9Vly>

<https://youtu.be/NtFgPngGvkc?si=T-z-KHvXBlaaiijo>

<https://youtu.be/NtFgPngGvkc?si=q72eSoaKRhP3Uexm>

https://youtu.be/VYkx24S6c_Y?si=-2eh66LMg_woBl4d

playlist 2

https://youtu.be/6JGzwl2pNfo?si=11lCIA_uoH_nzNI

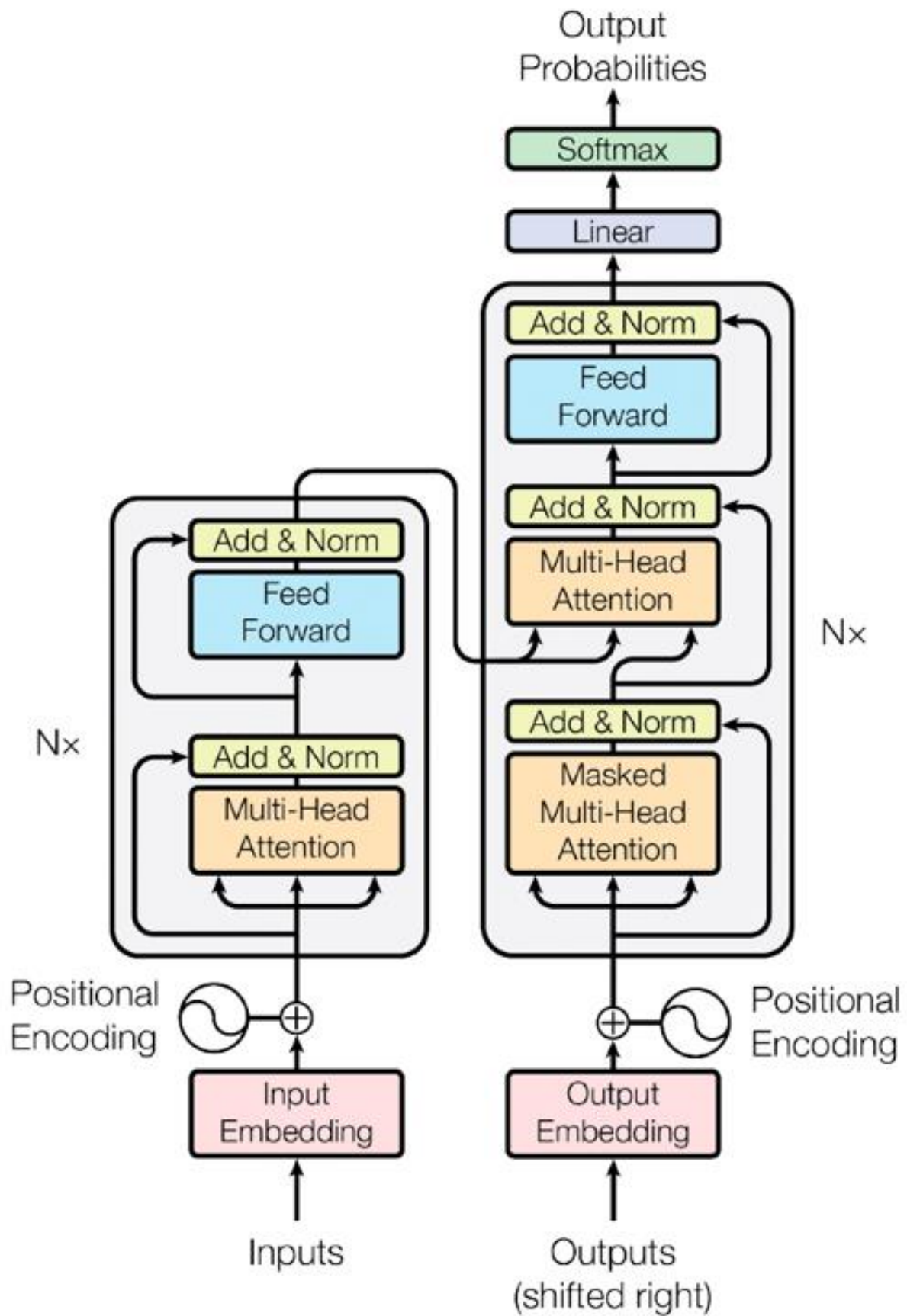
playlist 3

https://youtu.be/hEDTLsL_JnQ?si=i8eNLpkx9sOte3pm

playlist 4

https://youtu.be/Ff8KU1dyJMA?si=KOAW9ULssz0-ep_6

<https://youtu.be/1vYxr2TmkLA?si=yLy82SPobeN-CGTz>



Fine Tuning

Fine-tuning is a process in machine learning where you take a pre-trained model—one that has already been trained on a large, general-purpose dataset—and adapt it to perform well on a specific task or domain by continuing its training on a smaller, task-specific dataset. The idea is that the pre-trained model has already learned useful features, patterns, and representations from the large dataset, so instead of starting training from scratch, you only need to adjust its weights slightly to specialize it for your new problem. This saves computational resources, reduces the need for massive amounts of data, and often leads to better performance. For example, a large language model like GPT might first be pre-trained on billions of words from books, articles, and websites. To make it excellent at legal document classification, you fine-tune it using a smaller dataset of legal texts so it can better understand the language and structure of legal writing. Fine-tuning can be full (updating all weights), partial (freezing some layers and only updating the final ones), or even parameter-efficient (using methods like LoRA or adapters to add small modules without changing the main weights). It is widely used in NLP, computer vision, speech recognition, and many other areas to transfer knowledge from general training to specific tasks.

<https://youtu.be/Mg-wOaX7WWw?si=8pkvJa6Mjz-zZDyk>

<https://www.geeksforgeeks.org/deep-learning/what-is-fine-tuning/>

https://youtu.be/H-oCV5brtU4?si=P2CWln_qCICKEiwU

Tasks

Task 0 : project structure

Create folder called **Cellula_1week_[your name]** and inside it create two folders , first with name **RNN** , second with name **LSTM**,

Task 1 : RNN Training

- Train an RNN model on the dataset provided in the link below and place your code in the RNN folder.
- Along with your code, include a PDF file containing the results of your training.
- Ensure that the F1 score is used as the evaluation metric.
- The trained model must achieve an F1 score of at least 75% ($F1 \geq 0.75$)
- Bonus will be awarded for including graphs and a confusion matrix.

Task 2 : LSTM Training

- Train an LSTM model on the dataset provided in the link below and place your code in the LSTM folder.
- Along with your code, include a PDF file containing the results of your training.
- Ensure that the F1 score is used as the evaluation metric.
- The trained model must achieve an F1 score of at least 85% ($F1 \geq 0.85$)

- **Bonus will be awarded for including graphs and a confusion matrix.**

